

Maps, Geocoding, and Location-Based Services

Contents

- Add Google Map to Android application
- Google Map methods
- Access phone's position

Add Google Map to Android application

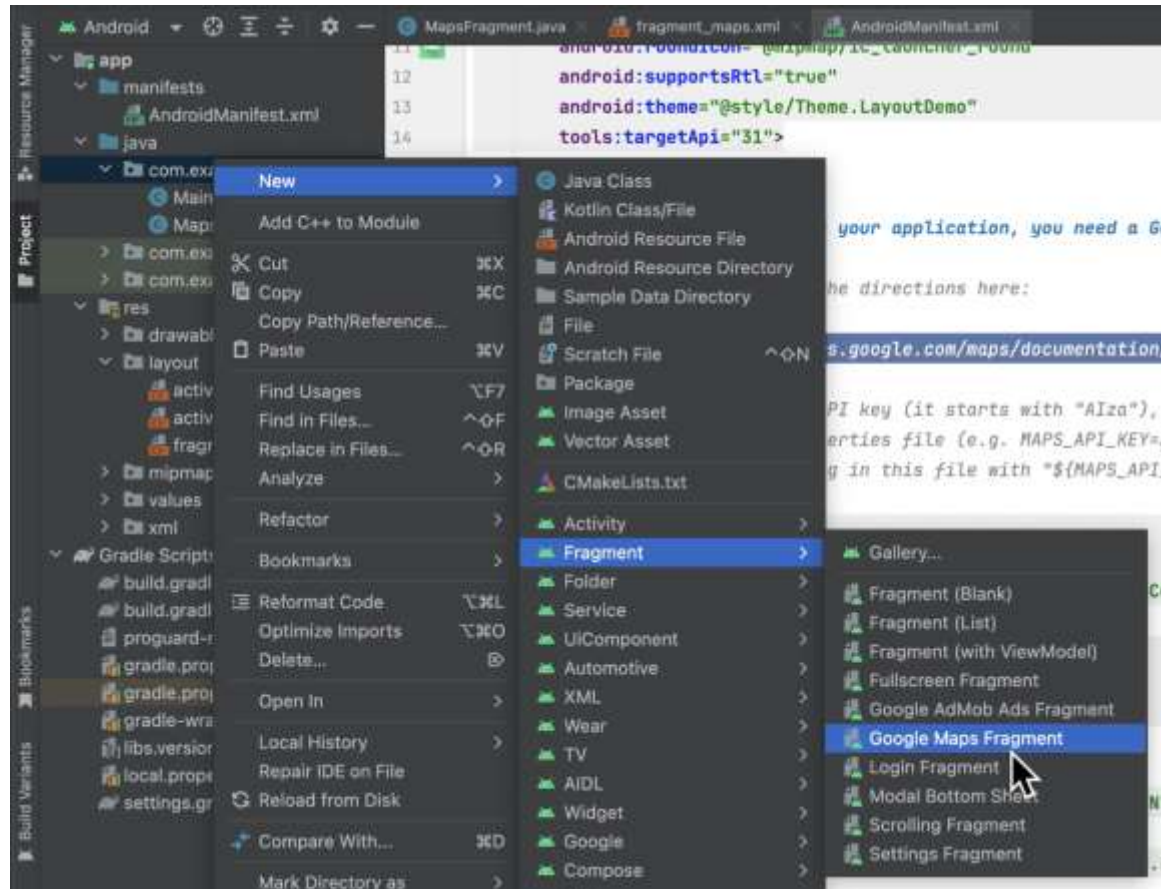
- Add Google Play Services to project
- Add MapFragment
- Get an API key
- Add permission to AndroidManifest.xml
- Wait for map to be ready

Step 1: Add Google Play Services to project

- add Google Play to project in app's build.gradle file

```
dependencies {  
    implementation fileTree(dir: 'libs', include: ['*.jar'])  
    implementation 'com.google.android.gms:play-services:18.0.0'  
}
```

2. Add MapFragment



Step 2: Add MapFragment

- Google Maps API provides a fragment class named MapFragment for displaying a map within an activity.

```
<?xml version="1.0" encoding="utf-8"?>
<fragment xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/map"
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MapsFragment" />
```

Step 2: Add MapFragment

After add fragment, some codes is generated as below
Include the direction link and meta-data

```
<!--
```

TODO: Before you run your application, you need a Google Maps API key.

To get one, follow the directions here:

<https://developers.google.com/maps/documentation/android-sdk/get-api-key>

Once you have your API key (it starts with "AIza"), define a new property in your project's local.properties file (e.g. MAPS_API_KEY=Aiza...), and replace the "YOUR_API_KEY" string in this file with "\${MAPS_API_KEY}".

```
-->
```

```
<meta-data
```

```
    android:name="com.google.android.geo.API_KEY"
```

```
    android:value="YOUR_API_KEY" />
```

Step 2: Add MapFragment

Go to the link from the direction from manifest file

<https://developers.google.com/maps/documentation/android-sdk/get-api-key>

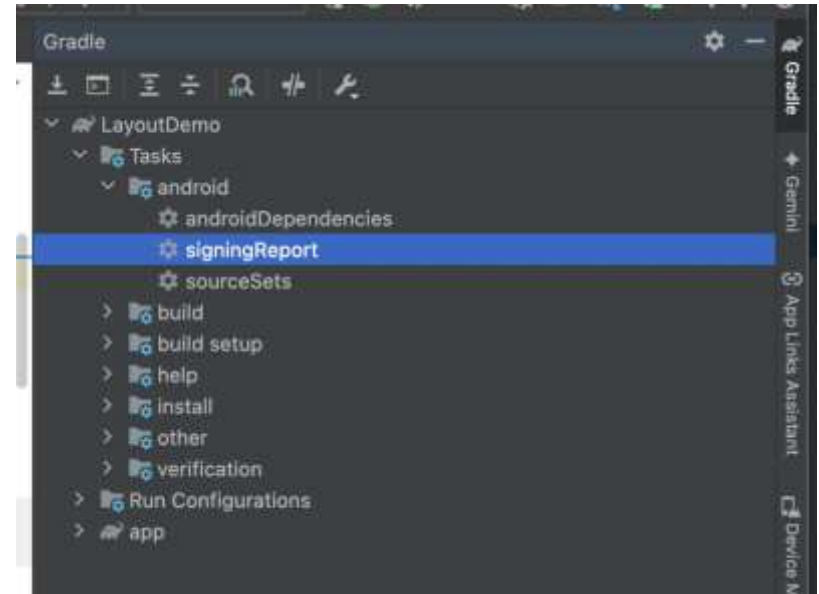
Go to the google console to create project, put SHA-1 (step 3)

Step 3: Get an API key

Google won't allow you to fetch map data without an API key.

Get SHA1 key

By running signingReport from android studio.



```
-----  
Variant: debugAndroidTest  
Config: debug  
Store: /Users/loanbui/.android/debug.keystore  
Alias: AndroidDebugKey  
MD5: 49:49:4D:ED:B7:F9:2A:BF:6F:E6:44:CC:19:76:2D:C9  
SHA1: 31:F4:38:58:01:65:D8:1E:47:1D:6B:3B:4E:E9:B7:8E:9E:5A:73:8C  
SHA-256: C9:C2:B1:63:50:36:9B:B8:82:BA:22:71:17:9E:7D:1E:BB:F6:E8:8A:0F:60:13:A8:AF:A6:99:E6:19:89:FB:DC  
Valid until: Wednesday, December 14, 2050
```

Step 3: Get an API key

- **Other way to get SHA1 key, open a Terminal and find the file debug.keystore:**

Windows (new): C:\Users\USERNAME\.android

Windows (old): C:\Documents and Settings\USERNAME\.android

Linux: /home/USERNAME/.android/

Mac: /Users/USERNAME/.android/

In the terminal or command prompt, cd to that directory, then type and run:

```
keytool -list -v -keystore debug.keystore
```

(it asks for a password, so just press Enter)

Find the line with your "Certificate fingerprint" for "SHA-1". It should contain a long string in this format. Copy it down.

```
BD:2B:3F:4B:.....
```

Step 3: Get an API key

- **Second: Go to the Google APIs developer console:**

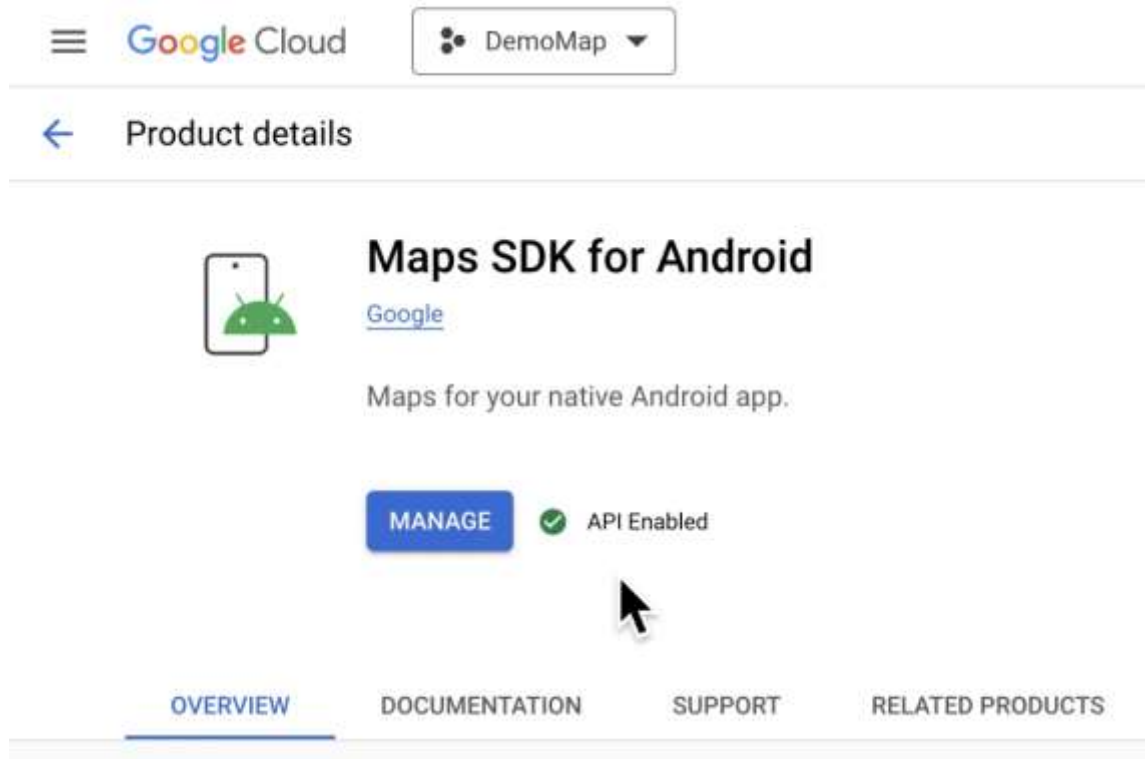
- Create project

Link: <https://code.google.com/apis/console/>

- Enable API
- choose Android Key
- paste in the SHA-1 key you got from the previous slide

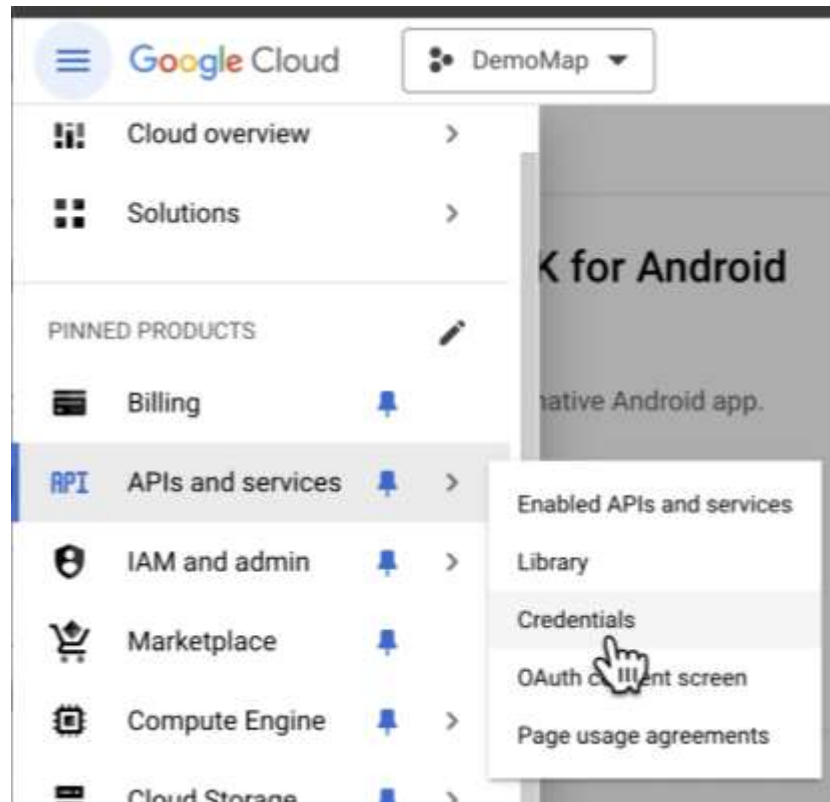
Step 3: Get an API key

- After create project
And enable API



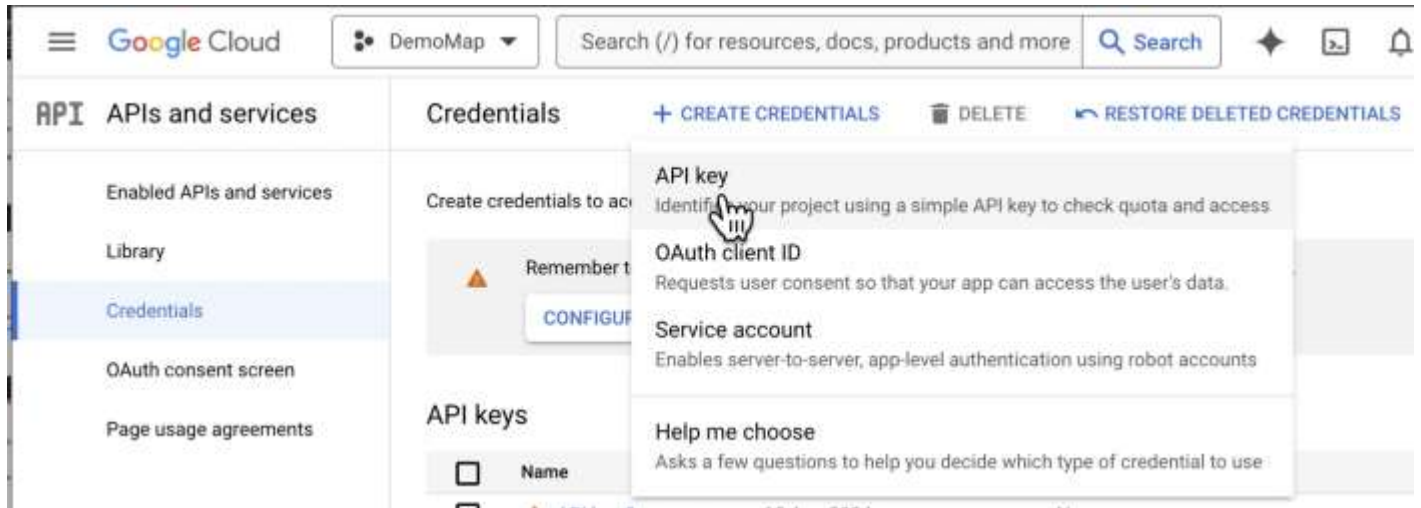
Step 3: Get an API key

- Cancel billing and select “Credentials” from menu



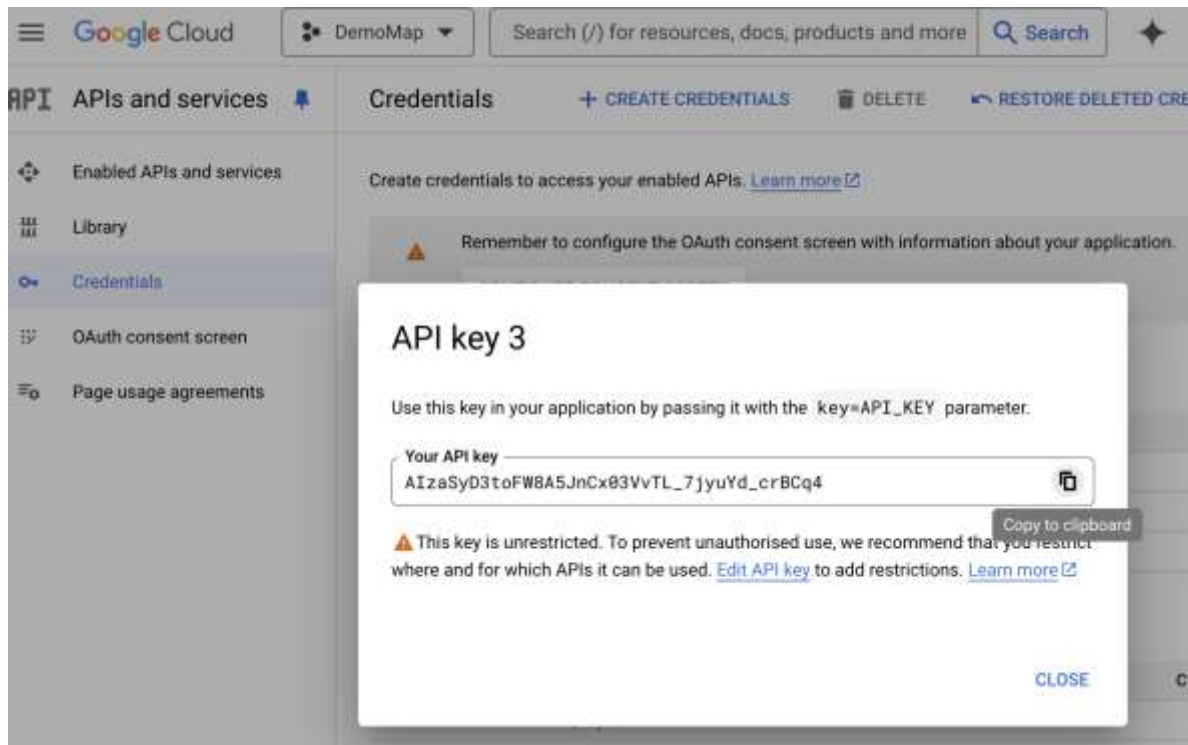
Step 3: Get an API key

- Create API key



Step 3: Get an API key

- Key generated
- Copy it and paste to
The manifest file



Step 3: Get an API key

Key is patsed to meta-data

```
<!--
```

TODO: Before you run your application, you need a Google Maps API key.

To get one, follow the directions here:

<https://developers.google.com/maps/documentation/android-sdk/get-api-key>

Once you have your API key (it starts with "AIza"), define a new property in your project's local.properties file (e.g. MAPS_API_KEY=Aiza...), and replace the "YOUR_API_KEY" string in this file with "\${MAPS_API_KEY}".

```
-->
```

```
<meta-data
```

```
  android:name="com.google.android.geo.API_KEY"
```

```
  android:value="AIzaSyD3toFW8A5JnCx03VvTL_7jyuYd_crBCq4" />
```

Step 4. AndroidManifest.xml add permissions

- Because of privacy issues, to access phone's current location, must request permission in AndroidManifest.xml:

```
<manifest ...>
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION" />
  <uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION" />
  <application ...>
    ...
  </application>
</manifest>
```

Step 4. AndroidManifest.xml add permissions

- ACCESS_COARSE_LOCATION Permission:

Purpose: allows an app to access approximate location based on network sources such as cell towers and Wi-Fi. The location obtained is less precise than using GPS but consumes less power.

Typical Usage: Apps that require location services but do not need pinpoint accuracy may request this permission. Examples include weather only need to know which city or area you are in rather than your exact position.

- ACCESS_FINE_LOCATION Permission:

Purpose: allows an app to access precise location data, typically from GPS, but it can also include data from Wi-Fi and mobile networks to provide a highly accurate location.

Typical Usage: Apps that require precise location tracking, such as navigation apps (e.g., Google Maps) or apps with geotagging features.

Step 5: Wait for map to be ready

```
@Override  
public void onMapReady(GoogleMap googleMap) {  
    LatLng sydney = new LatLng( latitude: -34, longitude: 151);  
    googleMap.addMarker(new MarkerOptions().position(sydney).title("Marker in Sydney"));  
    googleMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));  
}
```

GoogleMap methods

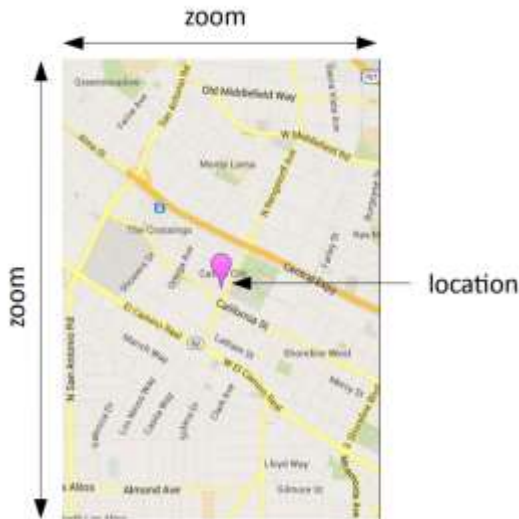
- **placing items on the map:**
 - addCircle, addGroundOverlay, addMarker, addPolygon, addPolyline, addTileOverlay
 - clear - Removes all markers, polylines/polygons, overlays
- **manipulating the camera:**
 - getCameraPosition, moveCamera, animateCamera, stopAnimation
- **map settings and appearance:**
 - setBuildingsEnabled, setIndoorEnabled, setMapType, setPadding, setTrafficEnabled

GoogleMap methods

- **snapshot - take a screen shot of the map as a bitmap**
- **event listeners:**
 - setOnCameraChangeListener, setOnMapClickListener, setOnMapLoadedCallback, setOnMapLongClickListener, setOnMarkerClickListener, setOnMarkerDragListener, setOnMyLocationChangeListener

The map's camera

- The current viewing window of a map's camera is defined by:
 - target location (latitude/longitude), zoom (2.0 - 21.0),
 - bearing (orientation/rotation), and tilt (degrees)



Set camera in XML

- **Set initial map settings and camera position in the layout XML:**

<fragment ...

android:name="com.google.android.gms.maps.MapFragment"

android:id="@+id/ID"

map:cameraBearing="112.5"

map:cameraTargetLat="-33.796923"

map:cameraTargetLng="150.922433"

map:cameraTilt="30"

map:cameraZoom="13"

map:mapType="normal"

map:uiCompass="false"

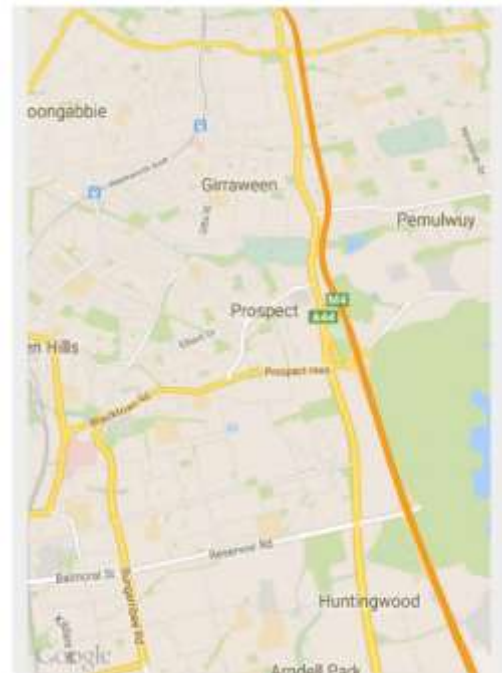
map:uiRotateGestures="true"

map:uiScrollGestures="false"

map:uiTiltGestures="true"

map:uiZoomControls="false"

map:uiZoomGestures="true" />



Set camera in Java code

- **CameraUpdateFactory methods:**

- newLatLng(new LatLng(lat, lng))
- newLatLngBounds(new LatLngBounds(SW, NE), padding)
- newLatLngZoom(new LatLng(lat, lng), zoom)
- newCameraPosition(CameraPosition)
- others:

```
LatLngBounds bounds = new LatLngBounds(  
    new LatLng(20, -130.0), // SW  
    new LatLng(55, -70.0)); // NE
```

```
map.moveCamera(CameraUpdateFactory.newLatLngBounds(bounds, 50));
```

Placing markers

- A GoogleMap object has an `addMarker` method that can let you put "push pin" markers at locations on the map.
 - The marker's methods return the marker, so you can chain them.
 - options: alpha, draggable, icon, position, rotation, title, visible, ...

```
map.addMarker(new MarkerOptions()  
    .position(new LatLng(40.801, -96.691))  
    .title("Lincoln, NE") );
```

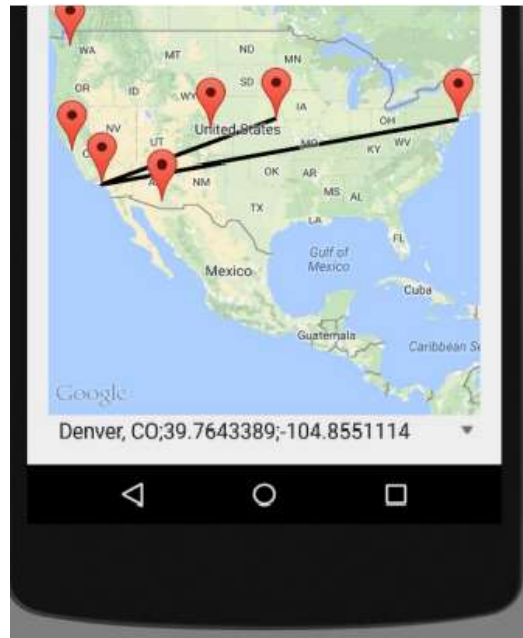
```
Marker mark = map.addMarker(new MarkerOptions() ...);  
mark.remove();
```



Lines and paths

- A **GoogleMap** object has an **addPolyline** method that can let you put lines between locations on the map.
 - options: color, visible, width, zIndex, ...

```
map.addPolyline(new PolylineOptions()  
    .add(new LatLng(40.801, -96.691)) // Lincoln, NE  
    .add(new LatLng(34.020, -118.412)) // Los Angeles, CA  
    .add(new LatLng(40.703, -73.980)) // New York, NY  
);  
Polyline polly = map.addPolyline(...); polly.remove();
```



Accessing phone's location

- **Android LocationManager gives you the phone's position:**
 - GPS provider provides highest accuracy
 - Network provider is a fallback in case GPS is disabled / not present

```
LocationManager locationManager = (LocationManager)
    getSystemService(Context.LOCATION_SERVICE);
Location loc = locationManager.getLastKnownLocation(
    locationManager.GPS_PROVIDER);
if (loc == null) {
    // fall back to network if GPS is not available
    loc = locationManager.getLastKnownLocation(
        locationManager.NETWORK_PROVIDER);
}
if (loc != null) {
    double myLat = loc.getLatitude();
    double myLng = loc.getLongitude();
    ...
    // other methods: getAltitude, getSpeed, getBearing, ...
```

Location update events

- Track user's movement by listening for location update events

```
LocationManager locationManager = (LocationManager)
    getSystemService(Context.LOCATION_SERVICE);

locationManager.requestLocationUpdates(
    LocationManager.GPS_PROVIDER, 0, 0,    // provider, min time/distance
    new LocationListener() {
        public void onLocationChanged(Location location) {
            // code to run when user's location changes
        }
        public void onStatusChanged(String prov, int stat, Bundle b){}
        public void onProviderEnabled(String provider) {}
        public void onProviderDisabled(String provider) {}
    }
);
```

Classwork

- Create a Google Map screen
- Create a marker showing your current location on the Map

References

- CS 193A Stanford course - Marty Stepp and Stanford Computer Science
- <https://developers.google.com/maps/documentation/android-sdk/map-with-marker>